ASYNC TASK QUEUE PROJECT PLAN

Building the Open-Source Python Solution for Hiring Impact

EXECUTIVE SUMMARY

This 6-month plan outlines development of an open-source Python async task queue system to attract backend engineering roles. The project addresses a real pain point (teams rebuilding task queue functionality) and showcases production-grade engineering.

WHY THIS PROJECT

The Problem:

Every Python team that outgrows Celery or doesn't want Redis dependency rebuilds task queue logic. Current solutions are either over-engineered (Celery) or too simple (RQ).

Why It Gets You Hired:

✓ Core strengths: async architecture, reliability patterns, production discipline

✓ Solves a real problem every backend team encounters

✓ Community engagement (not just portfolio projects)

✓ Technical leadership signal

6-MONTH TIMELINE

Phase	Duration	Deliverables	Visibility
Month 1-2	Foundation	Core + SQL + retry logic	GitHub repo + README
Month 2-3	Features	Dead letters + examples	2-3 blog posts
Month 4-5	Growth	Outreach + benchmarks	HN launch + 500-1000 stars
Month 6	Positioning	Case study + hiring narrative	Job applications

MONTH 1-2: FOUNDATION PHASE

Goal: Build core async task queue with SQL persistence, retry logic, and clean API.

Starting Point:

Your event-driven-task-engine is the seed. Polish it, add production features, open-source it.

Week 1-2: Architecture & Design

- ■ Define API: task definition, task submission, worker registration
- ■ Design schema: PostgreSQL tables for tasks, workers, retries, dead letters
- ■ Plan observability: logging structure, metrics hooks, health checks
- ■ Write architecture doc explaining design choices

Week 3-4: Core Implementation

- ■ Task producer: submit task → store in DB → enqueue for processing
- ■ Worker pool: async worker threads, graceful shutdown, health heartbeat
- ■ Retry logic: exponential backoff, max retries, circuit breaker
- ■ SQL persistence: PostgreSQL backend (no in-memory dependency)

Week 5-6: Testing & Polish

- ■ Test suite: happy path, failures, retries, graceful shutdown
- ■ Type hints throughout (mypy strict mode)
- ■ Docker: Dockerfile + docker-compose for local dev
- ■ Example: FastAPI + simple task

Week 7-8: Launch & Documentation

- ■ Comprehensive README with architecture diagram
- ■ Comparison doc: vs Celery, RQ, dramatiq
- ■ GitHub Actions CI: lint, type check, test, coverage
- ■ Publish to PyPI v0.1.0

MONTH 2-3: FEATURES & CONTENT PHASE

Goal: Add production features, create content that attracts users and hiring attention.

Week 1-2: Production Features

- ■ Dead letter queue for failed tasks
- ■ Task routing to specific worker pools

- ■ Priority queues
- ■ Scheduled tasks (cron-like)

Week 3-4: Observability & Examples

- ■ Built-in metrics: counts, latency, error rates (Prometheus-compatible)
- ■ Structured logging: JSON correlation with task IDs
- ■ 3+ examples: FastAPI, Django, standalone

Blog Posts (Write 3):

- **#1:** Why We Built Another Task Queue (500 words)
- **#2:** Exactly-Once Task Execution in Python (800 words)
- **#3:** Async Processing Without Redis (600 words)

MONTH 4-5: GROWTH & VISIBILITY PHASE

Goal: Drive adoption and position yourself as creator of a credible tool.

Community Outreach:

- ■ Hacker News: 'Show HN: A task queue for Python teams'
- ■ Reddit r/Python: Thoughtful discussion, respond to questions
- ■ Python Slack communities: Share in #projects
- ■ Direct outreach: 10-15 FastAPI/async teams for feedback
- ■ Benchmarks: Performance vs Celery/RQ (honest numbers)

Metrics to Aim For:

- ✓ 500-1000 GitHub stars
- ✓ 100+ PyPI downloads per week
- ✓ 20-30 issues/comments (active community)
- ✓ 5-6 blog posts published

MONTH 6: HIRING POSITIONING PHASE

Goal: Prepare hiring narrative and apply to roles.

Your Story:

> I identified a recurring pattern: every Python team rebuilds async task queue logic when Celery becomes too much or Redis is unavailable. I built an open-source solution adopted by 100+ teams. It demonstrates how I approach production: reliability through idempotent design, observability from day one, systems that fail predictably.

Visibility Activities:

- ■ Update LinkedIn: mention project, link GitHub
- ■ Post retrospective: 'Lessons from 6 months building open-source'
- ■ Create case study (optional SaaS variant)

Job Applications:

- ■ Target: Backend Engineer, Infrastructure, Systems Engineer roles
- ■ Target companies: Stripe, Figma, Linear, Discord, Mercury
- ■ Pitch: Link project in resume/cover letter

TECH STACK

Python 3.11+ | FastAPI (optional HTTP) | asyncio + aiohttp | PostgreSQL + SQLAlchemy async + Alembic | Python logging (JSON) + Prometheus hooks | pytest + pytest-asyncio (85%+ coverage) | Docker + GitHub Actions

KEY DESIGN PRINCIPLES

1. Exactly-Once Semantics: Tasks execute once even if workers crash. Idempotency required.

2. No External Dependencies: PostgreSQL only. Deploy anywhere.

3. Observable from Day One: Structured logging, metrics, health endpoints.

4. Graceful Failure: Retry logic, dead letter queues, operator control.

5. Simple API: One function to submit tasks. No learning curve.

IMMEDIATE NEXT STEPS

This Week:

- ■ Review event-driven-task-engine code
- ■ Sketch schema: tasks, workers, retries, dead letters tables
- ■ Create GitHub repo (name ideas: taskpulse, conductor, dispatch)
- ■ Write initial README + architecture doc

Next 2 Weeks:

- ■ Build core: producer, worker pool, retry logic
- ■ Add PostgreSQL persistence
- ■ First test suite (happy path + failures)

Weeks 3-4:

- ■ Polish: Docker, GitHub Actions, type hints, docstrings
- ■ FastAPI example
- ■ Publish to PyPI

FINAL NOTES

This is not about viral success. It's about building something real that solves a real problem. Companies hire the person behind the project. Show discipline, ship regularly, answer questions, iterate. That's what matters.

Timeline is realistic: 20-30 hrs/week = 120-180 hrs/month. This plan needs 100-120 hrs/month focused work. Includes blog posts, community interaction, and shipping.

You have the skills. Your GitHub shows production discipline, async expertise, shipping mentality. This project amplifies what you already know.

Created: July 12, 2026